Master 2 **Machine Learning on Sequences**

Practical work 1:
Hidden Markov Models (HMM) are generative models

In this session, you will use a small part of a toolkit to generate artificial samples by exploiting trained models and using their generative modeling capacity.

The observations are pixel columns (of height 28 pixels) of handwritten digit images of the MNIST dataset.

Exemple of image digit of the MNIST dataset.

For the problem we are interested in, each image will be represented by a sequence of columns of pixels. The images in the MNIST database are of size 28 X 28 pixels, so the problem is to model sequences of length T=28 where each observation is a vector of 28 pixels coded on 256 gray levels. The observations are thus continuous vectors in $\mathbb{R}^{28}$. By nature, we should use a continuous HMM as a model. But we choose to simplify the programming exercise by using semi-continuous HMM models.

Semi-continuous HMMs take continuous data as input but they model the problem in a discretized observation space. This is possible by introducing a preprocessing step that discretizes the continuous observations using a clustering algorithm, in our case we use the k-means algorithm implemented in the sklearn toolkit (sklearn.cluster module).

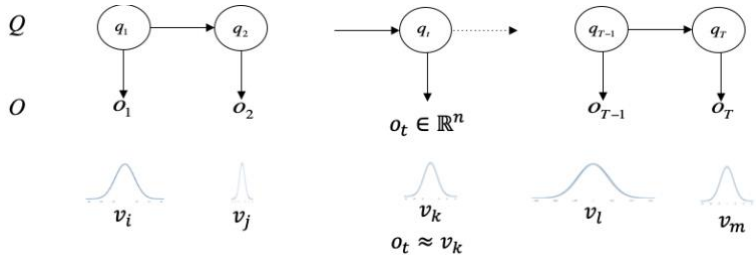
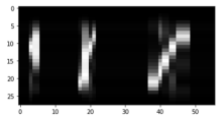
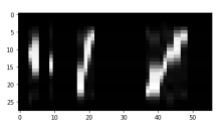


Illustration of a semi-continuous Hidden Markov Model.

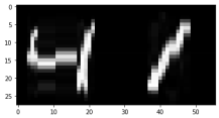
At the end of this preprocessing, we have a set of clusters (a "codebook"), and the columns of pixels are approximated by the closest cluster. This results in a discretization error that depends on the number of clusters. The figure below illustrates the discretization effect, and the corresponding discretized sequence, as a function of the number of clusters used to perform the discretization.



0 0 0 5 1 1 0 0 0 0 0 0 0 0 0 0 0 4 7 7 5 2 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 4 4 4 6 9 9 5 5 2 2 2 2 0 0 0 0 0 0 0



1 1 1 19 2 2 1 1 1 8 1 1 1 1 1 1 1 5 5 2 6 16 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 12 5 5 5 4 8 19 19 6 6 16 16 1 1 1 1 1 1 1



2 2 2 21 8 5 0 0 0 0 0 21 21 21 21 21 21 6 4 33 13 11 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 39 6 6 36 7 32 40 40 13 1 11 11 2 2 2 2 2 2 2

Discretization effect using 10, 20, 50 clusters, and the corresponding to a discreet sequence.

Notice that the codebook is shared by the different HMM. In this exercise you can choose to use two different codebooks of different size.

1. Generate synthetic data with the trained HMM

Write a method that exploits the generative Hidden Markov Model property to generate synthetic digits images of size 28 x 28. The trained models are made available in a specific folder following the naming convention: *dir_name = Model_n_clusters_n_states*. Six different types of models are provided in six different folders, each containing the ten models of the ten-digit classes:

Model_50_5	*class_0_50_5_10000_1000*
	class_1_50_5_10000_1000
	…
	class_9_50_5_10000_1000
Model_50_10	*class_1_50_5_10000_1000*
	…
	class_9_50_5_10000_1000
Model_50_15	*…*
Model_100_5	
Model_100_10	
Model_100_15	

The trained codebook is made available in files with the naming convention:

Codebook_n_clusters_N_train

Two codebooks are made available:

Codebook_50_10000
Codebook_100_10000

The method will have the following interface, being a member of the HMM class

```
def GenerateSamples(self,N_samples,length):

    return seq_states, seq_observations
```

indications:

- use the uniform random generator *np.random.default_rng(.)*
- *seq_states and seq_observation are two dimensional numpy matrices of size (N_samples, length)*

Conclusion

- How do you find the quality of the generated samples?
- How would you qualify the imperfection of the generated data ?
- What could be the reasons of the quality observed?

2. Classify the synthetic data using Viterbi

- *Write a method that computes the log likelihood of the best hidden state path using the Viterbi algorithm, using the following interface.*

```
def Viterbi(self,obs_seq):

    return Log_likelihood
```

- *Then, compute the 10 log likelihood scores of the ten models for each generated observation sequence and vote for the best model that can be associated.*

- *Compute the Character Recognition Rate (CRR) and the Character Error Rate (CER) over the generated data.*

3. Classify the real data using Viterbi

- *Use the MNIST test dataset to test the recognition performance on the real data. Compute the CRR and the CER.*

Conclusion

- Is Viterbi a good classification method ?

Appendix

hmm.py Toolbox *main functionalities available*

This toolbox will be enriched by new functionalities during the lab sessions.

```
import hmm
```

hmm.load_codebook(self,file_name)

Load a trained codebook

input `file_name` : *name of the codebook to be loaded, it follows the naming convention:* ***Codebook_n_states_N_train***
Available codebooks: Codebook_5_10000, Codebook_10_10000, Codebook_15_10000

Output `codebook` *: a codebook as computed by scikitlearn.cluster.KMeans*
`codebook.labels_` : *the best cluster associated to the training examples*
codebook.cluster_centers_ : the cluster centers

Call example

```
codebook = hmm.load_codebook(file_name)
```

hmm.HMM(self,n_states,n_clusters,dir_name="none",file_name="none")

By default, when no file_name is provided, creates a left-right discrete HMM with the specified number of states and discreet observations. Probabilities are initialized with uniform distribution.
When a file_name and a dir_name is provided the method loads the HMM from a file following the naming conventions.
dir_name = Models_n_clusters_n_states
file_name = Class_i_n_cluster_n_states_N_train_N_valid

input `n_states` : *number of the:*
`n_clusters` :
`dir_name` : i
`file_name` :

Output `hmm_model` *: an instance of the class HMM for the given class of pattern (digit)*

Call example

```
model = hmm.HMM(n_states,n_clusters,dir_name,file_name)
```

hmm.Codebook(X, N, D, n_clusters, LOAD_CODEBOOK=False,VERBOSE=True)

1- Compute a codebook with the scikitlearn KMEAN clustering alogorithm, store it with the discretized training data, following the naming conventions
or
2- Load a codebook and the discrete training data
return: the discrete training data and the codebook (scikit learn KMeans class instance)

input `X` : *name of the codebook to be loaded, it follows the naming convention*
`D` : *length of the sequences*
`n_clusters` : *number of clusters*
`LOAD_CODEBOOK=False` *if True the codebook is loaded, otherwise it is computer*
`VERBOSE=True` *Viewing the discretization effect*

Output `codebook` *: a codebook as computed by scikitlearn.cluster.KMeans*

Call example

```
LOAD_CODEBOOK = True
codebook = hmm.Codebook(X, N_train, D, n_clusters, LOAD_CODEBOOK =
LOAD_CODEBOOK)
```